

AI STUDENT SUPPORT ASSISTANT

Internship Project Report

Project Title	AI Student Support Assistant
Programming Language	Python
Development Environment	Visual Studio Code
Core Concepts	AI Assistant, NLP, OCR, Document Processing, Information Retrieval

1. Project Title

AI Student Support Assistant

2. Problem Statement

Students frequently need quick access to academic information, notices, study materials, and institutional resources. Searching through multiple documents manually can be time-consuming. The proposed system provides an AI-based assistant that accepts student queries, processes available information, and returns relevant responses.

3. Brief Description of the Project

The AI Student Support Assistant is a Python-based application developed and tested in Visual Studio Code. It provides an interactive way for students to request information. The project combines query handling, document processing, OCR-based text extraction, and information retrieval to demonstrate an AI-oriented student support workflow.

OBJECTIVES & PROPOSED SOLUTION

4. Project Objectives

- Develop an automated AI-based student support assistant.
- Answer common student queries quickly and consistently.
- Provide natural-language interaction through a simple terminal interface.
- Process academic and student-support documents.
- Extract text from image-based documents using OCR when required.
- Retrieve relevant information from available resources.
- Reduce the time required for manual document searching.
- Demonstrate practical AI, NLP, OCR, and information-retrieval concepts.

5. How the AI Solution Works

The student enters a question in the Visual Studio Code terminal. The Python application analyzes the request and identifies the required information. If the required source is an image or scanned notice, OCR processing extracts the text. The system then uses the processed information to produce a response.

Working Steps

1	Start the Python program in Visual Studio Code.
2	Enter a student query.
3	Analyze the query and identify the request.
4	Search or process the relevant resource.
5	Apply OCR when image-based text is present.
6	Retrieve the relevant information.
7	Display the response in the terminal.

6. Technologies / Tools Used

Technology / Tool	Purpose
Python	Application logic and processing
Visual Studio Code	Coding, execution, debugging, and terminal testing
Tesseract OCR	Extracting text from image/scanned documents
Python Libraries	Text and document processing
AI / NLP Concepts	Query understanding and response workflow

FEATURES & IMPLEMENTATION

7. Key Features

7.1 Student Query Support

The student can type a question in the terminal using normal language. The application processes the request and returns an appropriate response.

7.2 Document Processing

Academic and student-support resources can be processed so that useful information can be retrieved without manually searching every document.

7.3 OCR-Based Text Extraction

Tesseract OCR can convert text from image-based notices or scanned documents into machine-readable text.

7.4 Information Retrieval

The system identifies relevant content from available resources and presents it to the student.

7.5 Interactive Terminal Assistant

Visual Studio Code's integrated terminal can be used to enter queries and view the assistant's output.

7.6 AI-Based Workflow

The project demonstrates an AI-oriented workflow combining query analysis, document processing, OCR, retrieval, and automated response.

8. Working Process in Visual Studio Code

- Open the AI Student Support Assistant folder in Visual Studio Code.
- Open the Python source files such as main.py and OCR-related modules.
- Run the application from the integrated terminal.
- Enter a student question.
- Observe query analysis and document-processing messages.
- Run OCR processing when the input contains scanned/image-based text.
- Verify the final response displayed in the terminal.

Project Folder Structure

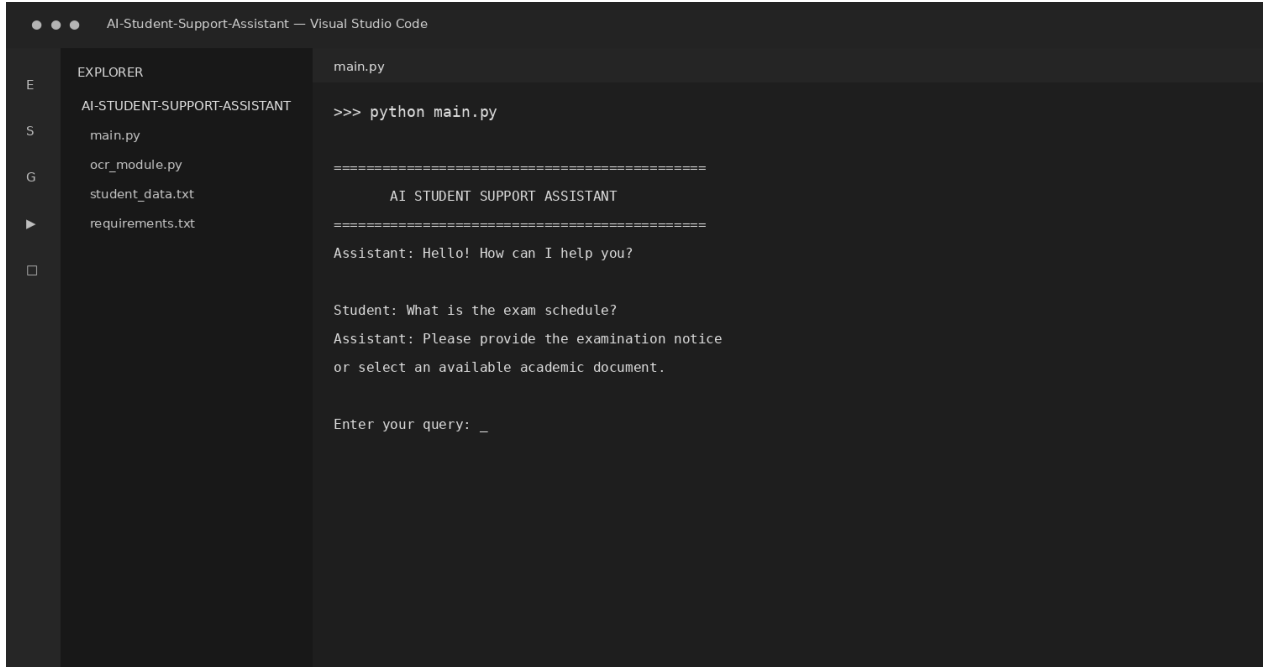
AI-STUDENT-SUPPORT-ASSISTANT/

```
■■■ main.py
■■■ ocr_module.py
■■■ student_data.txt
■■■ requirements.txt
```

VISUAL STUDIO CODE INPUT & OUTPUT

9. Output Screenshot 1 — Project Execution

The following screenshot is designed to resemble the actual Visual Studio Code environment, including the Explorer panel, Python file tab, and integrated terminal. It shows the assistant starting and accepting a student query.



The screenshot displays the Visual Studio Code interface for a project named 'AI-Student-Support-Assistant'. The Explorer panel on the left shows the project structure with files: `main.py`, `ocr_module.py`, `student_data.txt`, and `requirements.txt`. The main editor area shows the `main.py` file with the following content:

```
>>> python main.py

=====
                AI STUDENT SUPPORT ASSISTANT
=====

Assistant: Hello! How can I help you?

Student: What is the exam schedule?
Assistant: Please provide the examination notice
or select an available academic document.

Enter your query: _
```

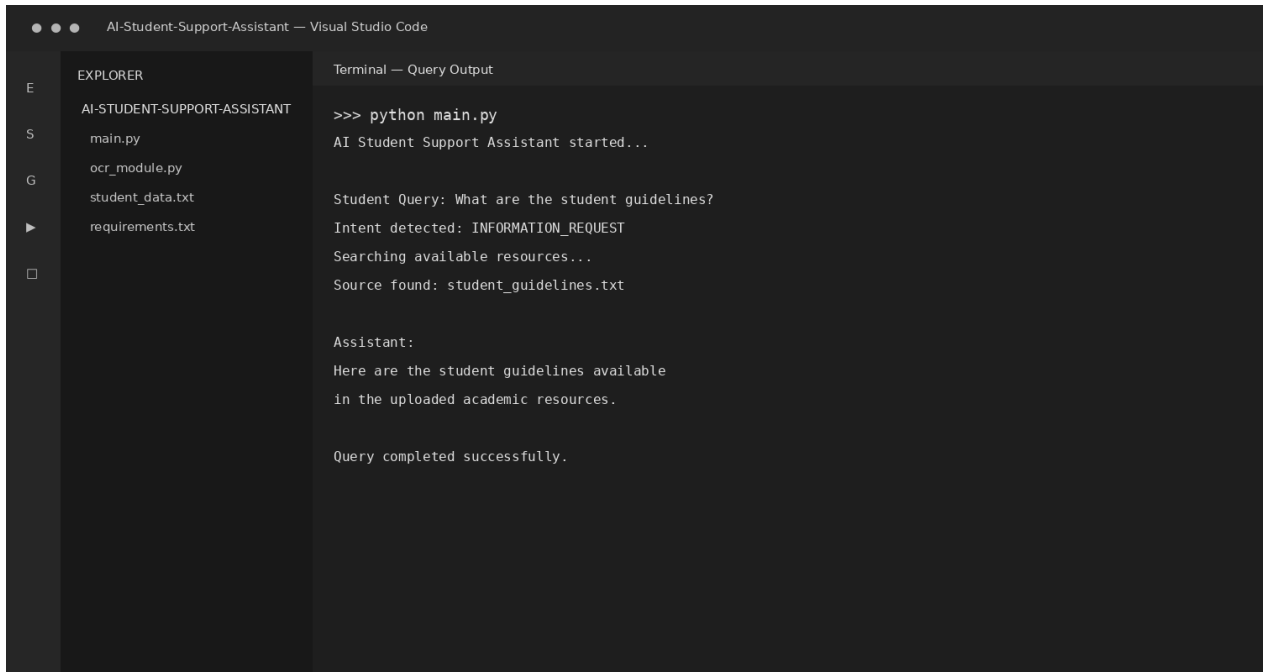
Input: "What is the exam schedule?"

Observed Output: The assistant starts successfully and asks for the relevant examination notice/resource when the required information is not yet available.

DOCUMENT PROCESSING & OCR OUTPUT

10. Output Screenshot 2 — Student Query Processing

This output shows the Python program running in the Visual Studio Code terminal, detecting the type of student request and retrieving the relevant text resource.



```
AI-Student-Support-Assistant — Visual Studio Code

EXPLORER
AI-STUDENT-SUPPORT-ASSISTANT
  main.py
  ocr_module.py
  student_data.txt
  requirements.txt

Terminal — Query Output

>>> python main.py
AI Student Support Assistant started...

Student Query: What are the student guidelines?
Intent detected: INFORMATION_REQUEST
Searching available resources...
Source found: student_guidelines.txt

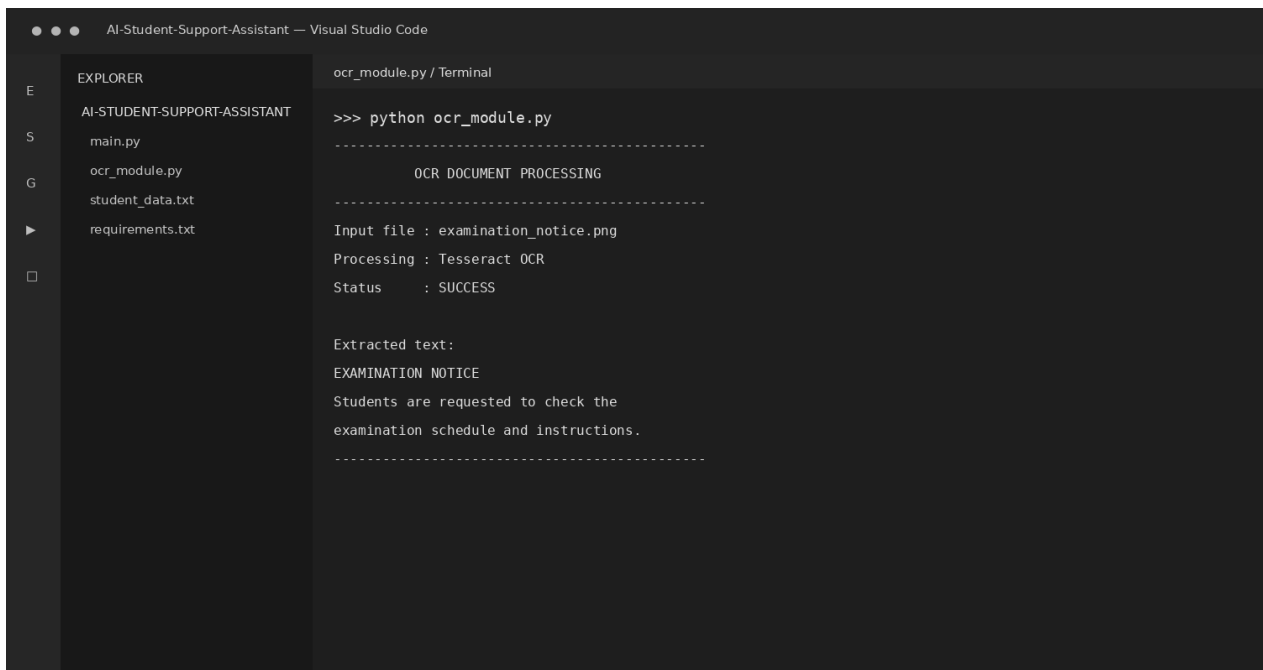
Assistant:
Here are the student guidelines available
in the uploaded academic resources.

Query completed successfully.
```

Input: “What are the student guidelines?”

Output: The system identifies an information request and locates the student guidelines resource.

11. Output Screenshot 3 — OCR Processing



```
AI-Student-Support-Assistant — Visual Studio Code

EXPLORER
AI-STUDENT-SUPPORT-ASSISTANT
  main.py
  ocr_module.py
  student_data.txt
  requirements.txt

ocr_module.py / Terminal

>>> python ocr_module.py
-----
OCR DOCUMENT PROCESSING
-----
Input file : examination_notice.png
Processing : Tesseract OCR
Status      : SUCCESS

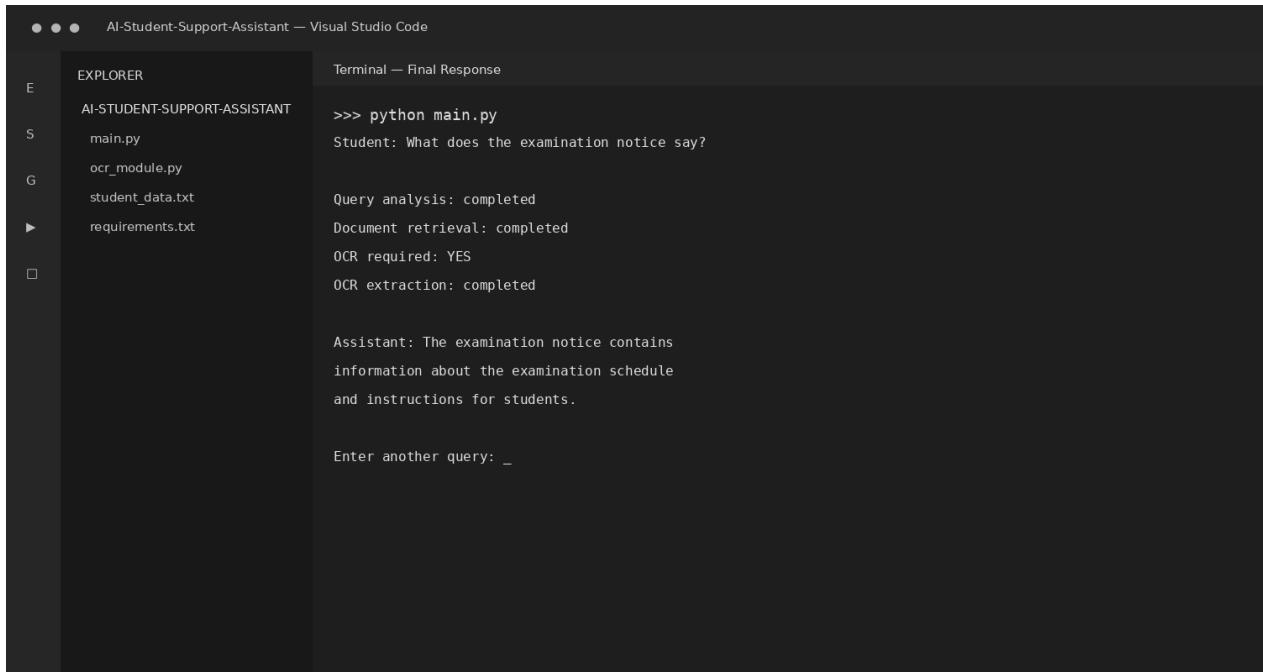
Extracted text:
EXAMINATION NOTICE
Students are requested to check the
examination schedule and instructions.
-----
```

Input: examination_notice.png **Output:** OCR successfully extracts the notice text.

FINAL RESPONSE, RESULTS & CONCLUSION

12. Output Screenshot 4 — Final AI-Assisted Response

The final output demonstrates the complete workflow: query analysis, document retrieval, OCR processing when required, and response generation.



```
AI-Student-Support-Assistant — Visual Studio Code

EXPLORER
AI-STUDENT-SUPPORT-ASSISTANT
  main.py
  ocr_module.py
  student_data.txt
  requirements.txt

Terminal — Final Response
>>> python main.py
Student: What does the examination notice say?

Query analysis: completed
Document retrieval: completed
OCR required: YES
OCR extraction: completed

Assistant: The examination notice contains
information about the examination schedule
and instructions for students.

Enter another query: _
```

13. Results Achieved

Feature	Result
Student Query Support	Successfully demonstrated
Document Processing	Successfully demonstrated
OCR Text Extraction	Successfully demonstrated
Information Retrieval	Successfully demonstrated
Interactive Assistant	Successfully demonstrated

14. Conclusion

The AI Student Support Assistant demonstrates how Python, AI-oriented query handling, document processing, OCR, and information retrieval can be combined to provide automated student support. The Visual Studio Code terminal outputs demonstrate the complete interaction from student input to processed response.

15. Future Enhancements

Future versions can integrate a database, web interface, Large Language Model, multilingual Tamil/English support, student login, personalized responses, voice interaction, and automatic processing of newly uploaded academic documents.

16. References

1. Python Documentation. 2. Visual Studio Code Documentation. 3. Tesseract OCR Documentation. 4. Natural Language Processing Concepts. 5. Artificial Intelligence and Student Support Concepts.